量化交易进阶:第73天 – 策略表现监控（每日净值更新+风控日志体系）

原创 逻辑与流 逻辑与流 2026年3月4日 10:37 北京 听全文

> 策略层、风控层、执行层、异常恢复都已具备，缺的是可观测性。机构量化系统永远有一个核心模块：Strategy Monitor。本文构建每日净值记录、实时回撤计算、风控日志、自动风险告警与绩效统计的机构级监控框架。

你现在已经有：

- 策略层
- 风控层
- 执行层
- 异常恢复

但缺一件非常关键的东西：

可观测性（Observability）

机构量化系统永远有一个核心模块：

Strategy Monitor（策略监控系统）

今天我们构建：

1. 每日净值记录系统
2. 实时回撤计算
3. 风控日志记录
4. 自动风险告警
5. 绩效统计计算

一、监控系统的核心目标

监控系统解决 4 个问题：

问题	解决方式
今天赚多少钱?	净值记录
是否异常回撤?	实时回撤计算
是否触发风控?	风控日志
是否超出风险预算?	自动报警

二、每日净值记录模块

NAV 记录器

```
import pandas as pd
import os
from datetime import datetime

class NAVMonitor:

    def __init__(self, file="nav_history.csv"):
        self.file = file
        if not os.path.exists(file):
            pd.DataFrame(columns=["date", "nav"]).to_csv(file, index=False)

    def record(self, nav):
        df = pd.read_csv(self.file)
        today = datetime.now().date()
        df = df[df["date"] != str(today)]  # 防止重复
        df.loc[len(df)] = [today, nav]
        df.to_csv(self.file, index=False)
```

三、实时回撤监控（核心风险指标）

机构最关注：

最大回撤（MDD）

计算回撤



千问 AI 双 11，Token Plan 低至5折
阿里云
查看

```
def compute_drawdown(nav_series):

    cummax = nav_series.cummax()
    drawdown = nav_series / cummax - 1
    max_dd = drawdown.min()

    return drawdown.iloc[-1], max_dd
```

集成到 Monitor

```
class PerformanceMonitor(NAVMonitor):

    def check_risk(self):

        df = pd.read_csv(self.file)
        nav_series = df["nav"]

        current_dd, max_dd = compute_drawdown(nav_series)

        print(f"当前回撤: {current_dd:.2%}")
        print(f"历史最大回撤: {max_dd:.2%}")

        return current_dd, max_dd
```

四、风控日志系统（非常关键）

真正的机构不会只记录交易日志。

他们还记录：

- 每次风控裁剪
- 每次仓位上限触发
- 每次止损触发
- 每次异常保护

风控日志类

```
import logging

risk_logger = logging.getLogger("RISK")
risk_logger.setLevel(logging.INFO)

handler = logging.FileHandler("risk.log")
risk_logger.addHandler(handler)
```

在风控模块中加入日志

```
class RiskController:

    def apply(self, w_target):

        w_safe = {}

        for s, w in w_target.items():
            if w > 0.05:
                risk_logger.info(f"{s} 触发单票上限 5%")
                w_safe[s] = 0.05
            else:
```

```
                w_safe[s] = w

        return w_safe
```

五、自动风险报警机制

例如：

- 当日亏损 > 3%
- 总回撤 > 15%
- 单票暴露 > 10%

```
def risk_alert(nav_monitor):

    df = pd.read_csv(nav_monitor.file)
    nav_series = df["nav"]

    daily_return = nav_series.pct_change().iloc[-1]

    if daily_return < -0.03:
        send_wechat_alert("⚠ 今日亏损超过3%")

    _, max_dd = compute_drawdown(nav_series)

    if max_dd < -0.15:
        send_wechat_alert("⚠ 最大回撤超过15%")
```

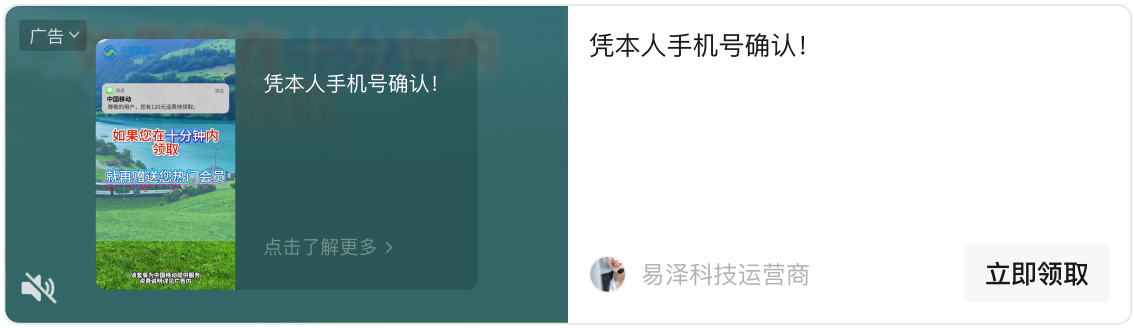
六、机构级完整监控流程

```
交易结束
   ↓
更新 NAV
   ↓
计算回撤
   ↓
检查风险阈值
   ↓
记录风控日志
   ↓
触发报警
```

七、每日绩效统计（进阶）

可以计算：

- 年化收益
- 夏普比率



- 波动率
- 卡玛比率

示例：年化收益 & 夏普

```
import numpy as np

def performance_stats(nav_series):

    returns = nav_series.pct_change().dropna()

    annual_return = (nav_series.iloc[-1] / nav_series.iloc[0]) ** (252/len(nav_series)) - 1
    annual_vol = returns.std() * np.sqrt(252)
    sharpe = annual_return / annual_vol

    return annual_return, sharpe
```

八、实盘中最重要的 3 个指标

机构实盘通常只盯三件事：

1. 净值曲线
2. 当前回撤
3. 风险暴露

模型 alpha 排在后面。

九、你现在的系统结构

```
Strategy Engine
    ↓
Risk Engine
    ↓
Execution Engine
    ↓
Monitoring Engine   ← 今天完成
```

这四个合起来，才叫：

完整的量化交易系统

十、为什么大多数人失败？

因为他们：

- 有策略
- 有回测
- 甚至有实盘

但没有：

监控系统

没有监控 = 黑箱运行。

结论

把净值、回撤、风控日志和报警都做齐，系统才可观测、可追溯、可告警，实盘才能放心跑。

量化交易入门指南 · 目录 ≡

〈上一篇
量化交易进阶:第72天 – 异常处理与重启机制（实盘断连恢复/异常自动报警）

下一篇 〉
量化交易进阶:第74天 – 自动复盘报告（每日自动生成量化策略报告PDF）

阅读 68



寒冬没有退路，合理分配每份资源才能守住营地。

无尽冬日–3亿人都在玩

立即体验